

Estudo Independente 03 — Especificação e Implementação de um Analisador Léxico

Componente curricular: GRP00543 — Linguagens Formais e Compiladores (UNIJUÍ, 2026/2º)

Professor: Marcos Ronaldo Melo Cavalheiro

Estudante(s):

Eduardo Manhabosco

Data: 25/09/2026

Linguagem: Gambiarra (projeto do TP1). Implementação manual em Python, módulo `re`.

A linguagem Gambiarra tem programa e blocos com `{ }`, declaração `remendo x = valor;` (o tipo vem do valor), saída `mostra(...)`, entrada `pega(x)`, decisão `se/senao`, repetição `enquanto` e os literais lógicos `firme/quebrado`.

```
// Teste 01: declaração, atribuição e saída (enunciado §8).
gambiarra {
    remendo fita = 10;           // inteiro (deduzido do valor)
    remendo cola = 2.5;         // real
    remendo ligado = firme;     // lógico

    fita = fita + 5;
    cola = cola * 2;

    mostra("fita: ", fita);
    mostra("cola: ", cola);
    mostra("ligado: ", ligado);
    mostra("Sistema Gambiarra iniciado");
}
/* Saída esperada do programa:
fita: 15
cola: 5
ligado: firme
Sistema Gambiarra iniciado
*/
```

Problema 1 — Especificação formal dos tokens

Notação: expressões regulares no estilo JFlex/Python: `|` = união, `*` = zero ou mais, `+` = um ou mais, `?` = opcional, `[a-z]` = classe de caracteres, `[^x]` = qualquer caractere menos x, `\.` = o ponto literal. *Atenção:* nos slides da Aula 3, `+` significa união; aqui, união é `|`.

Alfabeto: $\Sigma = \{a-z, A-Z\} \cup \{0-9\} \cup \{ _ " . , ; () \{ \} + - * / \% = ! < > \& | \backslash \} \cup \{ \text{espaço, tab, quebra de linha} \}$. Letras acentuadas só valem dentro de texto e de comentário.

a) Palavras reservadas

```
RESERVADA = gambiarra | remendo | mostra | pega | se | senao | enquanto | firme | quebrado
```

Cada palavra é uma linguagem de uma palavra só, e a união delas é regular. São todas minúsculas e **diferenciam maiúsculas**: `Se` é identificador. Todas também casam com a ER de identificador; o conflito é resolvido no Problema 2a.

b) Identificadores

```
LETRA = [a-zA-Z_]      DIGITO = [0-9]
ID    = LETRA (LETRA | DIGITO)*    = [a-zA-Z_][a-zA-Z0-9_]*
```

Começa obrigatoriamente com letra ou `_` (assim `2abc` nunca é identificador) e continua com letras, dígitos ou `_`. Sem acento, porque o C gerado não aceita acento em nome de variável. Aceita `fita`, `_ok`, `total_2`; rejeita `2abc`, `preço`, `a-b` (este último vira três tokens: `a`, `-`, `b`).

c) Números inteiros e reais

```
NUM_INT  = DIGITO+      = [0-9]+
NUM_REAL = DIGITO+ \. DIGITO+  = [0-9]+\.[0-9]+
```

O inteiro é uma sequência de um ou mais dígitos. O real tem **exatamente um ponto**, com pelo menos um dígito de cada lado. `3.4.5` não é gerado pela ER (teria dois pontos), nem `5.` ou `.5`. O sinal negativo não faz parte do número: `-5` são dois tokens (`MENOS` e `NUM_INT`), e o parser trata o menos unário. Assim, `x-5` continua sendo uma subtração.

d) Operadores aritméticos, relacionais e lógicos

```
ARITMETICO = \+ | - | \* | / | %
RELACIONAL = == | != | >= | <= | > | <
LOGICO     = && | \|\| | !
```

`+`, `*` e `|` são escapados com `\` porque são operadores da própria notação de ER. Os de dois caracteres aparecem antes dos de um (ver Problema 2b).

e) Atribuição, delimitadores e pontuação

```
ATRIB = =
DELIMITADOR = \( | \) | \{ | \} | ; | ,
TEXTO = " ( [^"\\\n] | \\[nt"\\] )* "
```

`=` guarda um valor e `==` compara (Problema 2b). Parênteses agrupam expressões e condições; chaves delimitam o programa e os blocos; `;` termina cada comando; `,` separa os itens do `mostra`. O texto fica entre aspas duplas, não pode quebrar linha e aceita os escapes `\n \t \" \\`.

f) Espaços, quebras de linha e comentários

```
BRANCO      = [ \t\r\n]+
COMENT_LINHA = // [^\n]*
COMENT_BLOCO = /\* ( [^*] | \*+ [^*/] )* \*+ /      (no código: /\*[^\s\S]*?\/)
```

São reconhecidos e **descartados** (não viram token). O comentário de linha vai de `//` até o fim da linha. O de bloco vai de `/*` até o **primeiro** `*/` e pode ter várias linhas. A ER formal descreve "qualquer coisa que não contenha `*/`"; no código, o quantificador preguiçoso `*?` faz o mesmo. As quebras de linha dentro dos comentários também são contadas para a posição.

AFD de dois tokens (ligação com a Aula 3)

Identificador, estados {q0, q1}, inicial q0, final {q1}:

δ	letra ou <code>_</code>	dígito	outro
$\rightarrow$ q0	q1	— (rejeita)	— (rejeita)

* q1	q1	q1	— (fim do token)
------	----	----	------------------

Número real, estados {q0, q1, q2, q3}, inicial q0, final {q3} (o inteiro é aceito em q1):

δ	dígito	.	outro
→ q0	q1	—	—
q1 (aceita NUM_INT)	q1	q2	— (fim)
q2	q3	—	—
* q3	q3	— (segundo ponto: erro)	— (fim)

Simulação de 3.4.5: q0 → q1 → q2 → q3 → sem transição. O léxico não aceita e acusa "número malformato". Simulação de 10.5: q0 → q1 → q1 → q2 → q3 (final): aceita.

Problema 2 — Resolução de conflitos e tratamento léxico

O léxico tem uma **tabela de regras** (*nome*, *ER*). Em cada posição do texto, as regras são testadas **em ordem** e a **primeira** que casa ganha (é o que o `re.match` do Python faz, e também a alternância `|`: `re.match("=|==", "==")` devolve `=`). A ordem da tabela resolve os conflitos abaixo.

a) Prioridade de reconhecimento: palavra reservada × identificador

`se`, `enquanto` e as outras reservadas também são palavras da linguagem de ID. Em vez de uma regra por reservada, usamos a técnica clássica (Aho, cap. 3): casar o lexema com a ER de **identificador** e depois consultar a tabela `PALAVRAS_RESERVADAS`. Se o lexema estiver lá, o token é a palavra reservada; senão, é ID. Vantagens: vale o maior casamento (`senaozinho` é ID inteiro, e não `senao` + `zinho`), e acrescentar uma palavra reservada é só uma linha na tabela.

```
def criar_token(nome, lexema, linha, coluna):
    if nome == "ID":
        tipo = PALAVRAS_RESERVADAS.get(lexema, TipoToken.ID) # reservada? senão ID
        return Token(tipo, lexema, None, linha, coluna)
```

b) Maior casamento (longest match)

Entre as regras que casam numa posição, o lexema certo é o **mais longo**: `>=` é um token só, e não `>` seguido de `=`. Geradores como o Flex fazem isso sozinhos. No léxico manual, garantimos a mesma coisa pela **ordem** da tabela: os operadores de dois caracteres (`==` `!=` `>=` `<=` `&&` `||`) vêm antes dos de um (`=` `!` `>` `<`), e `NUM_REAL` vem antes de `NUM_INT`. Para números e identificadores, o próprio `+/*` da ER já consome o máximo, e as ERs terminam com `(?![\w.])`, que exige que o número acabe ali. Sem isso, `2abc` viraria `NUM(2) ID(abc)` em vez de erro.

Sem o operador longo primeiro, `x == 1` sairia como `ID ATRIB ATRIB NUM`, e o parser acusaria um erro que não existe.

c) Controle de posição (linha e coluna)

O léxico mantém três contadores: `pos` (índice no texto), `linha` (começa em 1) e `coluna` (começa em 1). Cada token (e cada erro) é criado com a linha e a coluna do **início** do lexema. Depois, os contadores avançam: se o lexema não tem quebra de linha, `coluna += len(lexema)`; se tem (espaços com `\n`,

comentário de bloco), `linha` soma o número de quebras, e a `coluna` passa a ser o número de caracteres depois da última quebra, mais 1. Assim a posição continua certa depois de comentários de várias linhas.

Problema 3 — Implementação

Arquivos: `tokens.py` (tipos de token e palavras reservadas), `erros.py` (`ErroLexico` com linha e coluna) e `lexico.py` (o analisador). Uso: `python lexico.py arquivo.gam`.

- **a) Entrada:** lê um arquivo de texto (UTF-8; aceita arquivos com BOM).
- **b) Saída:** imprime a sequência de tokens, um por linha, com **linha:coluna, classe e lexema**. Números já são convertidos (`int/float`) no campo de valor do token.
- **c) Erros:** regras de erro na tabela reconhecem número malformato (`2abc`, `3.4.5`, `5.`), identificador com caractere fora de Σ (`preço`), texto sem fechar aspas, escape inválido, comentário de bloco sem fechar e, por último, o "pega-tudo" `.:` qualquer outro caractere (`@`, `#`, `&` sozinho) é erro. Cada erro registra **linha, coluna e lexema**, e a leitura **prossegue** depois do lexema problemático. Todos os erros aparecem no fim.

Código completo de `lexico.py` (a tabela `REGRAS` é o coração do léxico; os outros dois arquivos estão no `.zip`):

```
"""
lexico.py – 1ª estação: o analisador léxico (scanner).

Lê o código-fonte caractere por caractere e devolve a lista de tokens.

Como funciona (a "cascata" do separador de moedas):
Em cada posição do texto, as REGRAS abaixo são testadas EM ORDEM, e a
primeira que casar ganha. O pedaço casado (o lexema) "cai no buraco" do seu
tipo de token. Depois o dedo anda para depois do lexema e tudo recomeça.

Três cuidados que o enunciado e o EI03 cobram:
1. Maior casamento (longest match): '==' vem ANTES de '=' na lista; senão
   '=' cairia no buraco errado e viraria dois '='. (EI03 P2b)
2. Palavra reservada × identificador: "se" casa com a regra de identificador
   e depois consultamos PALAVRAS_RESERVADAS. (EI03 P2a)
3. Erro léxico: as regras de ERRO (no fim da lista, ou logo depois da regra
   certa que falhou) guardam o erro com linha, coluna e lexema, e a leitura
   CONTINUA. (EI03 P3c)

Uso isolado (só o léxico, útil para o EI03):
python lexico.py testes/06_lexico_todos_tokens.gam
"""

import re
import sys

from erros import ErroLexico
from tokens import PALAVRAS_RESERVADAS, TipoToken, Token

# -----
# Tabela de regras: (nome da regra, expressão regular).
# A ORDEM IMPORTA: é testada de cima para baixo.
# Nomes que são um TipoToken viram token; os outros têm tratamento especial
# (ignorar ou registrar erro) na função analisar().
# -----
REGRAS = [
    # --- ignorados: espaços, quebras de linha e comentários ---
    ("ESPACO", r"[ \t\r\n]+"),
```

```

("COMENT_LINHA",      r"//[^\n]*"),                # // até o fim da linha
("COMENT_BLOCO",      r"/\*[^\s\S]*?\/"),            # /* ... */ (o '?' pega o PRIMEIRO */)
("ERRO_COMENT",        r"/\*[^\s\S]*"),              # /* sem */ até o fim do arquivo

# --- texto: aspas duplas, sem quebra de linha, escapes \n \t \" \\ ---
("TEXTO",             r'"(?:\\[nt\\]|^[^\n])*"'),
("ERRO_ESCAPE",        r'"(?:\\.|^[^\n])*"'),          # fechou, mas tem escape inválido (ex.: \q)
("ERRO_TEXTO",         r'"^[^\n]*'),                 # abriu aspas e a linha acabou

# --- números: o (?![w.]) exige que o número termine ali ---
# sem ele, "2abc" viraria NUM(2) ID(abc) e "3.4.5" viraria NUM(3.4) ...
("NUM_REAL",           r"[0-9]+\.[0-9]+(?![w.])"),    # 10.5 (real ANTES do inteiro)
("NUM_INT",            r"[0-9]+(?![w.])"),            # 10
("ERRO_NUMERO",        r"[0-9][w.]*|\.[0-9][w.]*"),  # 2abc, 3.4.5, 5., .5

# --- identificador (e palavra reservada, decidida depois) ---
# (?!w) recusa um nome colado a letra acentuada: "preço"
("ID",                r"[A-Za-z_][A-Za-z0-9_]*(?!w)"),
("ERRO_ID",           r"w+"),                        # nome com acento ou outro caractere fora de Σ

# --- operadores de DOIS caracteres: antes dos de um (maior casamento) ---
("IGUAL_IGUAL",        r"=="),
("DIFERENTE",          r"!="),
("MAIOR_IGUAL",        r">="),
("MENOR_IGUAL",        r"<="),
("E_LOGICO",           r"&&"),
("OU_LOGICO",          r"||"),

# --- operadores e delimitadores de UM caractere ---
("ATRIB",              r"="),
("NAO_LOGICO",         r"!"),
("MAIOR",              r">"),
("MENOR",              r"<"),
("MAIS",               r"+"),
("MENOS",              r"-"),
("VEZES",              r"\*"),
("DIVIDE",             r"/"),
("RESTO",              r"%"),
("ABRE_PAREN",         r"\("),
("FECHA_PAREN",        r"\)"),
("ABRE_CHAVE",         r"\{"),
("FECHA_CHAVE",        r"\}"),
("PONTO_VIRGULA",      r";"),
("VIRGULA",            r","),

# --- "pega-tudo": qualquer outro caractere é ERRO (nunca vira token) ---
("ERRO_CARACTERE",     r"."),

```

```
]
```

```
# Compila uma vez só (mais rápido e deixa o laço principal limpo).
```

```
REGRAS_COMPILADAS = [(nome, re.compile(padrao)) for nome, padrao in REGRAS]
```

```
# Regras que só consomem texto, sem gerar token.
```

```
IGNORADAS = {"ESPACO", "COMENT_LINHA", "COMENT_BLOCO"}
```

```
# Mensagem de cada regra de erro.
```

```

MENSAGENS_ERRO = {
    "ERRO_COMENT":      "comentário de bloco '/' aberto e nunca fechado com '*/'",
    "ERRO_ESCAPE":      "texto com sequência de escape inválida (valem só \\n, \\t, \\\" e \\\\)",
    "ERRO_TEXTO":       "texto sem fechar aspas (o texto não pode quebrar linha)",
    "ERRO_NUMERO":       "número malformado (inteiro: 10; real: 10.5; nome não começa com dígito)",
    "ERRO_ID":           "identificador com caractere inválido (use só letras sem acento, dígitos e _)",
    "ERRO_CARACTERE":    "caractere que não pertence à linguagem",
}

```

```

def analisar(codigo):
    """
    Varre o código e devolve (tokens, erros).

    tokens: lista de Token, terminando SEMPRE com um token FIM.
    erros: lista de ErroLexico (vazia se deu tudo certo).
    """
    tokens = []
    erros = []
    pos = 0          # o "dedo": índice do caractere atual no texto
    linha = 1        # controle de posição (EI03 P2c)
    coluna = 1

    while pos < len(codigo):
        # Testa as regras em ordem; fica a PRIMEIRA que casar.
        for nome, regex in REGRAS_COMPILADAS:
            casamento = regex.match(codigo, pos)
            if casamento:
                break
        # A última regra ('.') casa com qualquer caractere menos '\n', e o '\n'
        # é pego por ESPACO; então sempre há casamento aqui.
        lexema = casamento.group()

        if nome in IGNORADAS:
            pass
        elif nome.startswith("ERRO_"):
            erros.append(ErroLexico(f"{MENSAGENS_ERRO[nome]}: {mostrar_lexema(lexema)}", linha,
coluna))
        else:
            tokens.append(criar_token(nome, lexema, linha, coluna))

        # Anda o dedo e atualiza linha/coluna contando as quebras de linha do lexema.
        quebras = lexema.count("\n")
        if quebras:
            linha += quebras
            coluna = len(lexema) - lexema.rfind("\n")  # caracteres depois do último '\n', +1
        else:
            coluna += len(lexema)
        pos = casamento.end()

    tokens.append(Token(TipoToken.FIM, "", None, linha, coluna))
    return tokens, erros

def criar_token(nome, lexema, linha, coluna):
    """Transforma o lexema casado em Token, já convertendo o valor."""
    if nome == "ID":
        # Palavra reservada x identificador: consulta a tabela.
        tipo = PALAVRAS_RESERVADAS.get(lexema, TipoToken.ID)
        return Token(tipo, lexema, None, linha, coluna)
    if nome == "NUM_INT":
        return Token(TipoToken.NUM_INT, lexema, int(lexema), linha, coluna)
    if nome == "NUM_REAL":
        return Token(TipoToken.NUM_REAL, lexema, float(lexema), linha, coluna)
    if nome == "TEXT0":
        # Guardamos o texto SEM as aspas. Os escapes ficam como estão (\n
        # continua sendo barra + n), porque o C entende os mesmos escapes.
        return Token(TipoToken.TEXT0, lexema, lexema[1:-1], linha, coluna)
    return Token(TipoToken[nome], lexema, None, linha, coluna)

def mostrar_lexema(lexema, limite=30):
    """Lexema para a mensagem de erro: entre aspas simples e cortado se for longo."""
    primeira_linha = lexema.split("\n", 1)[0]
    if len(primeira_linha) > limite:
        primeira_linha = primeira_linha[:limite] + "..."

```

```

return f'{primeira_linha}'

def tabela_tokens(tokens):
    """Texto com um token por linha: posição, tipo e lexema (a saída do EI03, P3b)."""
    linhas = [f'{LIN:COL':<9}{TIPO':<15}LEXEMA"]
    for t in tokens:
        linhas.append(f'{f'{t.linha}:{t.coluna}':<9}{t.tipo.name:<15}{t.lexema}")
    return "\n".join(linhas)

# Rodando este arquivo sozinho: mostra os tokens e os erros léxicos de um arquivo.
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("uso: python lexico.py <arquivo.gam>")
        sys.exit(1)
    sys.stdout.reconfigure(encoding="utf-8")
    with open(sys.argv[1], encoding="utf-8-sig") as arquivo:      # -sig: ignora o BOM
        lista, problemas = analisar(arquivo.read())
    print(tabela_tokens(lista))
    for erro in problemas:
        print(erro)
    sys.exit(1 if problemas else 0)

```

Problema 4 — Casos de teste e evidências

a) Arquivo válido com todos os tipos de token — 06_lexico_todos_tokens.gam

```

// opcoes: --so-lexico
// Teste 06: todos os tipos de token válidos (EI03 P4a). Só o léxico roda.
/* comentário
de bloco */
gambiarra {
    remendo inteiro_1 = 42;
    remendo real_1 = 3.14;
    remendo _ok = firme || quebrado && !firme;
    se (inteiro_1 == 42 && inteiro_1 != 0) {
        mostra("texto com \"aspas\", \\ e \t tab\n", inteiro_1 % 5);
    } senao {
        pega(inteiro_1);
    }
    enquanto (real_1 >= 1.0 || inteiro_1 <= 0 || inteiro_1 > 1 || inteiro_1 < 2) {
        real_1 = real_1 / 2 - 1 + 0.5 * -1;
    }
}

```

Saída de `python lexico.py testes/06_lexico_todos_tokens.gam`:

LIN:COL	TIPO	LEXEMA
5:1	GAMBIARRA	gambiarra
5:11	ABRE_CHAVE	{
6:5	REMENDO	remendo
6:13	ID	inteiro_1
6:23	ATRIB	=
6:25	NUM_INT	42
6:27	PONTO_VIRGULA	;
7:5	REMENDO	remendo
7:13	ID	real_1
7:20	ATRIB	=
7:22	NUM_REAL	3.14
7:26	PONTO_VIRGULA	;
8:5	REMENDO	remendo

8:13	ID	_ok
8:17	ATRIB	=
8:19	FIRME	firme
8:25	OU_LOGICO	
8:28	QUEBRADO	quebrado
8:37	E_LOGICO	&&
8:40	NAO_LOGICO	!
8:41	FIRME	firme
8:46	PONTO_VIRGULA	;
9:5	SE	se
9:8	ABRE_PAREN	(
9:9	ID	inteiro_1
9:19	IGUAL_IGUAL	==
9:22	NUM_INT	42
9:25	E_LOGICO	&&
9:28	ID	inteiro_1
9:38	DIFERENTE	!=
9:41	NUM_INT	0
9:42	FECHA_PAREN	)
9:44	ABRE_CHAVE	{
10:9	MOSTRA	mostra
10:15	ABRE_PAREN	(
10:16	TEXTO	"texto com \"aspas\", \\ e \t tab\n"
10:52	VIRGULA	,
10:54	ID	inteiro_1
10:64	RESTO	%
10:66	NUM_INT	5
10:67	FECHA_PAREN	)
10:68	PONTO_VIRGULA	;
11:5	FECHA_CHAVE	}
11:7	SENAO	senao
11:13	ABRE_CHAVE	{
12:9	PEGA	pega
12:13	ABRE_PAREN	(
12:14	ID	inteiro_1
12:23	FECHA_PAREN	)
12:24	PONTO_VIRGULA	;
13:5	FECHA_CHAVE	}
14:5	ENQUANTO	enquanto
14:14	ABRE_PAREN	(
14:15	ID	real_1
14:22	MAIOR_IGUAL	>=
14:25	NUM_REAL	1.0
14:29	OU_LOGICO	
14:32	ID	inteiro_1
14:42	MENOR_IGUAL	<=
14:45	NUM_INT	0
14:47	OU_LOGICO	
14:50	ID	inteiro_1
14:60	MAIOR	>
14:62	NUM_INT	1
14:64	OU_LOGICO	
14:67	ID	inteiro_1
14:77	MENOR	<
14:79	NUM_INT	2
14:80	FECHA_PAREN	)
14:82	ABRE_CHAVE	{
15:9	ID	real_1
15:16	ATRIB	=
15:18	ID	real_1
15:25	DIVIDE	/
15:27	NUM_INT	2
15:29	MENOS	-
15:31	NUM_INT	1
15:33	MAIS	+
15:35	NUM_REAL	0.5

```

15:39 VEZES      *
15:41 MENOS      -
15:42 NUM_INT     1
15:43 PONTO_VIRGULA ;
16:5  FECHA_CHAVE }
17:1  FECHA_CHAVE }
18:1  FIM

```

Todas as categorias aparecem: as 9 palavras reservadas, ID, NUM_INT, NUM_REAL, TEXTO (com os 4 escapes), os 5 aritméticos, ATRIB, os 6 relacionais, os 3 lógicos e os 6 delimitadores. Os comentários de linha e de bloco foram descartados, e a posição continuou certa depois do comentário de 2 linhas (o programa começa em 5:1).

b) Arquivo com erros léxicos propositalis — 03_erro_lexico.gam

```

// Teste 03: erros léxicos (enunciado §7 e EI03 P4b). O léxico reporta TODOS e continua lendo.
gambiarra {
    remendo x = 10;
    mostra(x @ 2);
    remendo 2abc = 5;
    remendo y = 3.4.5;
    remendo preço = 1;
    x = x & 1;
    mostra("texto sem fechar");
    remendo z = 5.;
}
/* comentário de bloco que nunca fecha

```

Saída de `python lexico.py testes/03_erro_lexico.gam`:

LIN:COL	TIPO	LEXEMA
2:1	GAMBIARRA	gambiarra
2:11	ABRE_CHAVE	{
3:5	REMENDO	remendo
3:13	ID	x
3:15	ATRIB	=
3:17	NUM_INT	10
3:19	PONTO_VIRGULA	;
4:5	MOSTRA	mostra
4:11	ABRE_PAREN	(
4:12	ID	x
4:16	NUM_INT	2
4:17	FECHA_PAREN	)
4:18	PONTO_VIRGULA	;
5:5	REMENDO	remendo
5:18	ATRIB	=
5:20	NUM_INT	5
5:21	PONTO_VIRGULA	;
6:5	REMENDO	remendo
6:13	ID	y
6:15	ATRIB	=
6:22	PONTO_VIRGULA	;
7:5	REMENDO	remendo
7:19	ATRIB	=
7:21	NUM_INT	1
7:22	PONTO_VIRGULA	;
8:5	ID	x
8:7	ATRIB	=
8:9	ID	x
8:13	NUM_INT	1
8:14	PONTO_VIRGULA	;
9:5	MOSTRA	mostra

```

9:11      ABRE_PAREN      (
10:5      REMENDO        remendo
10:13     ID              z
10:15     ATRIB          =
10:19     PONTO_VIRGULA  ;
11:1      FECHA_CHAVE    }
13:1      FIM

```

Erro léxico [linha 4, coluna 14]: caractere que não pertence à linguagem: '@'
 Erro léxico [linha 5, coluna 13]: número malformato (inteiro: 10; real: 10.5; nome não começa com dígito): '2abc'
 Erro léxico [linha 6, coluna 17]: número malformato (inteiro: 10; real: 10.5; nome não começa com dígito): '3.4.5'
 Erro léxico [linha 7, coluna 13]: identificador com caractere inválido (use só letras sem acento, dígitos e _): 'preço'
 Erro léxico [linha 8, coluna 11]: caractere que não pertence à linguagem: '&'
 Erro léxico [linha 9, coluna 12]: texto sem fechar aspas (o texto não pode quebrar linha): '"texto sem fechar);'
 Erro léxico [linha 10, coluna 17]: número malformato (inteiro: 10; real: 10.5; nome não começa com dígito): '5.'
 Erro léxico [linha 12, coluna 1]: comentário de bloco '/*' aberto e nunca fechado com '*/': '/* comentário de bloco que nun...'

Os 8 erros foram reportados na mesma execução, cada um com linha, coluna e lexema, e os tokens válidos entre eles continuaram sendo reconhecidos: o léxico não parou no primeiro erro.

Referências

MENEZES, Paulo Blauth. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2011. Cap. 3.
 AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Pearson Addison-Wesley, 2007. Cap. 3.
 PRICE, Ana Maria de Alencar; TOSCANI, Simão Sirineo. *Implementação de linguagens de programação: compiladores*. 2. ed. Porto Alegre: Bookman, 2001. Cap. 2.2.
 UNIJUÍ. Plano de Ensino — Linguagens Formais e Compiladores, 2026. Slides das Aulas 02 e 03.
 PYTHON SOFTWARE FOUNDATION. *re* — *Regular expression operations*.
<https://docs.python.org/3/library/re.html>.

Declaração de uso de Inteligência Artificial generativa

Item	Declaração
Ferramenta	Claude Code (modelo Claude Opus 5.5, da Anthropic).
Finalidade	explicação dos conceitos (token, lexema, maior casamento, palavra reservada × identificador, geradores como Lex/Flex); levantamento das decisões do léxico, apresentadas como perguntas: as escolhas (alfabeto, formato dos números, identificadores, comentários, escapes) foram feitas pelo estudante ; escrita do código do analisador léxico, dos arquivos de teste e do rascunho deste documento.
Etapas	Problemas 1 a 4.
Verificação das ER e dos autômatos	testes de reconhecimento executados: arquivo com todos os tokens válidos (P4a) e arquivo com 8 erros propositais (P4b), com as saídas conferidas à mão, token a token, contra as ER do P1; teste isolado de <code>re.match("= ==", "==")</code> × <code>re.match("== =", "==")</code> para confirmar a necessidade da ordem; simulação manual do AFD de número real com 10.5 e 3.4.5; arquivo salvo com BOM (achado e corrigido durante os testes).

Fontes de validação	slides das Aulas 02 e 03; enunciado do TP1 (tabela de tokens da BIRL-Lite); documentação do módulo re. [PREENCHER: livros efetivamente consultados]
Alterações do estudante	definição de todas as escolhas do léxico. [PREENCHER: o que o estudante alterou no código ou no texto]